



**UNIVERSIDAD TÉCNICA DE MANABÍ**  
**FACULTAD DE CIENCIAS INFORMÁTICAS**  
**CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN**

**ABRIL DEL 2026 HASTA AGOSTO DEL 2026**

**TEMA:**

Estudio de caso - Sistema de Gestión de Incidentes (Help Desk)

**ESTUDIANTE:**

ZAMBRANO PINARGOTE EDGAR MOISÉS

**DOCENTE:**

ING. JORGE ARUN ZAMBRANO CEDEÑO

**MATERIA:**

DESARROLLO DE SISTEMAS INFORMÁTICOS

**NIVEL:**

QUINTO

**PARALELO:**

“A”

**FECHA:**

16-05-2026

## **Descripción**

## **Actores Principales**

### **Usuario Final**

Es la persona que reporta incidencias detectadas en el Data Center.

#### **Funciones:**

- Crear tickets
- Consultar estado de tickets
- Adjuntar evidencias
- Recibir notificaciones

### **Técnico de Soporte**

Responsable de diagnosticar y resolver incidentes.

#### **Funciones:**

- Visualizar tickets asignados
- Actualizar estados
- Registrar soluciones
- Escalar incidentes
- Agregar comentarios técnicos

### **Administrador de Sistemas**

Gestiona la operación general del Help Desk.

#### **Funciones:**

- Administrar usuarios y roles
- Configurar categorías
- Asignar técnicos
- Generar reportes
- Supervisar SLA y rendimiento

## **Requerimientos Funcionales**

### **Gestión de Tickets**

- Crear ticket de incidente
- Editar ticket
- Consultar ticket
- Cerrar ticket
- Reabrir ticket

### **Gestión de Fallos**

#### **Categorizar fallos:**

- Hardware
- Red
- Software

#### **Establecer prioridad:**

- Baja
- Media
- Alta
- Crítica

## **Gestión de Técnicos**

- Asignar técnico automáticamente o manualmente
- Reasignar técnico
- Escalar incidentes

## **Seguimiento**

- Registrar historial de cambios

Actualizar estados:

- Abierto
- En proceso
- Escalado
- Resuelto
- Cerrado

## **Notificaciones**

- Enviar alertas por correo o sistema
- Informar cambios de estado

## **Reportes**

- Reportes de incidencias
- Tiempo promedio de resolución
- Rendimiento de técnicos

## **Patrones de diseño**

### **Singleton**

El patrón singleton se utiliza para limitar la creación de una clase a un solo objeto. Esto es beneficioso cuando se necesita un objeto (y solo uno) para coordinar acciones en todo el sistema (Falcon, 2020).

El patrón singleton evitará lo siguiente dentro del sistema:

- Inconsistencia de datos
- Duplicación de recursos
- Consumo excesivo de memoria
- Conflictos en la administración de tickets

### **Observer**

Este patrón es una dependencia de uno a muchos entre objetos, de modo que cuando un objeto cambia de estado, se notifica a todos sus dependientes. Normalmente, esto se hace llamando a uno de sus métodos (Falcon, 2020).

Se utilizará para:

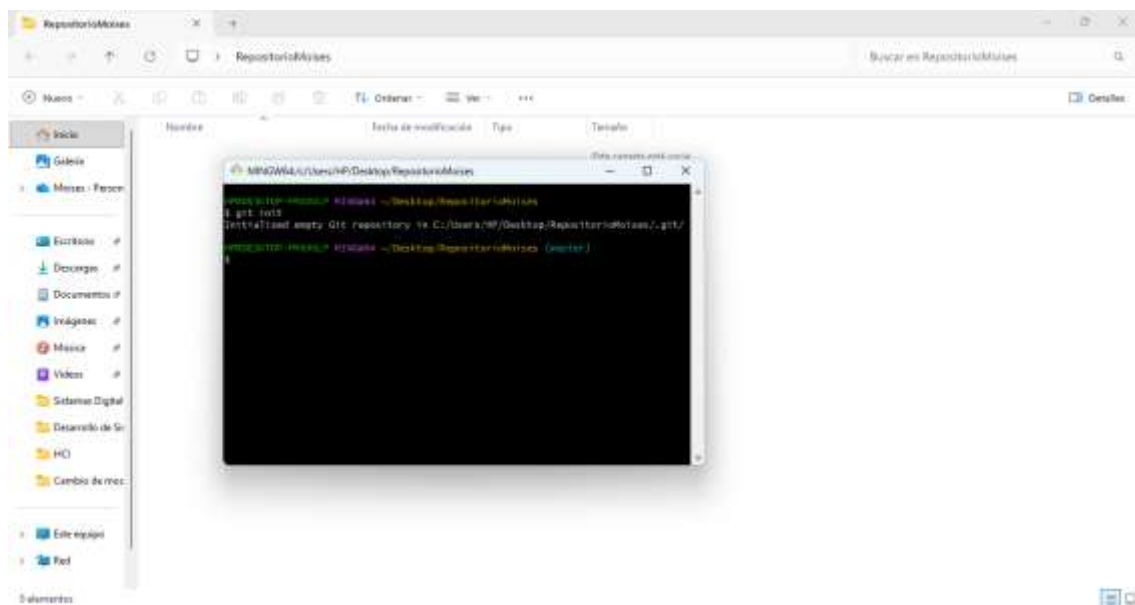
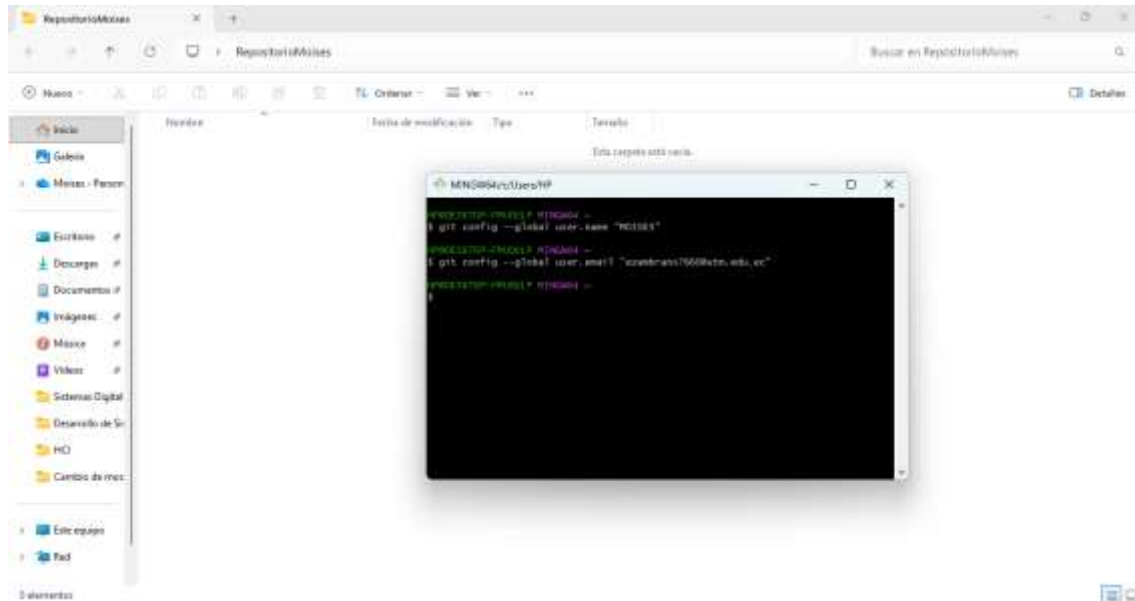
- Notificaciones de cambios en tickets
- Alertas automáticas
- Seguimiento de incidencias

### **Factory Method**

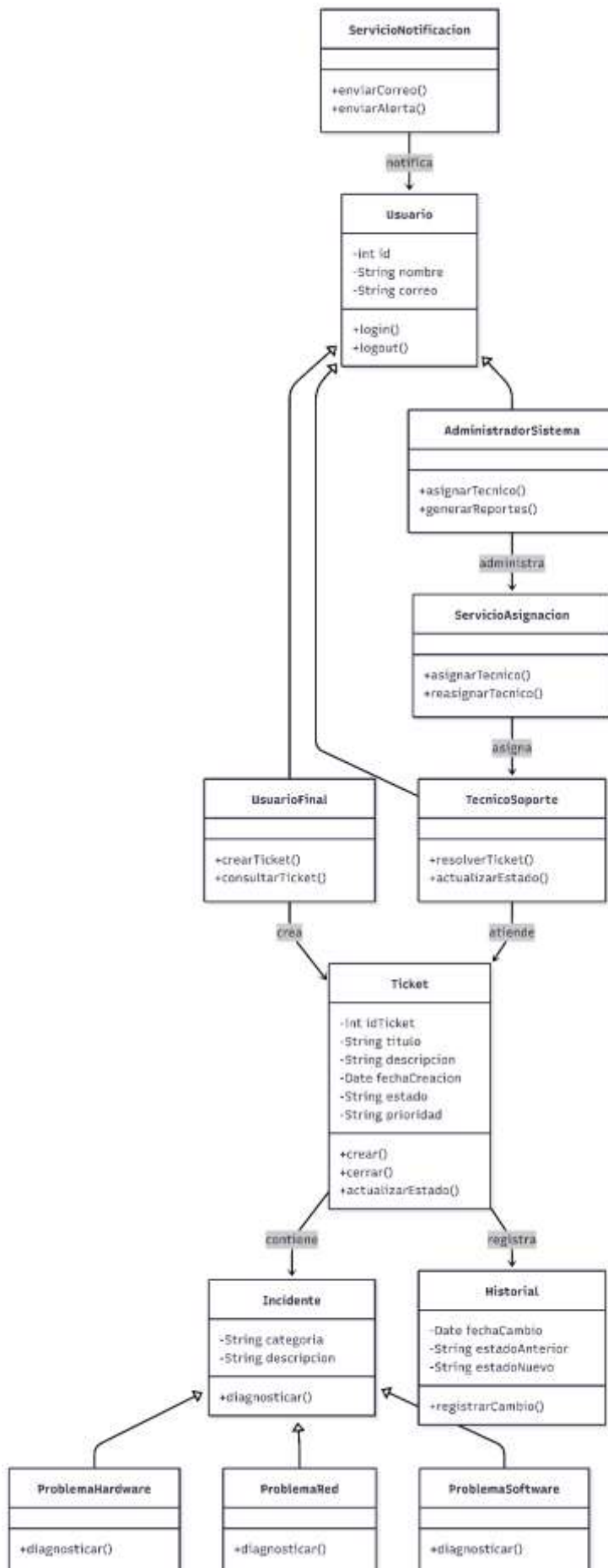
Proporciona una interfaz para crear objetos en una superclase, pero permite que las subclases alteren el tipo de objetos que se crearán. Proporciona instanciación de objetos implícita a través de interfaces comunes (Miriam & Miriam, 2026).

Se utilizará para crear distintos tipos de incidentes:

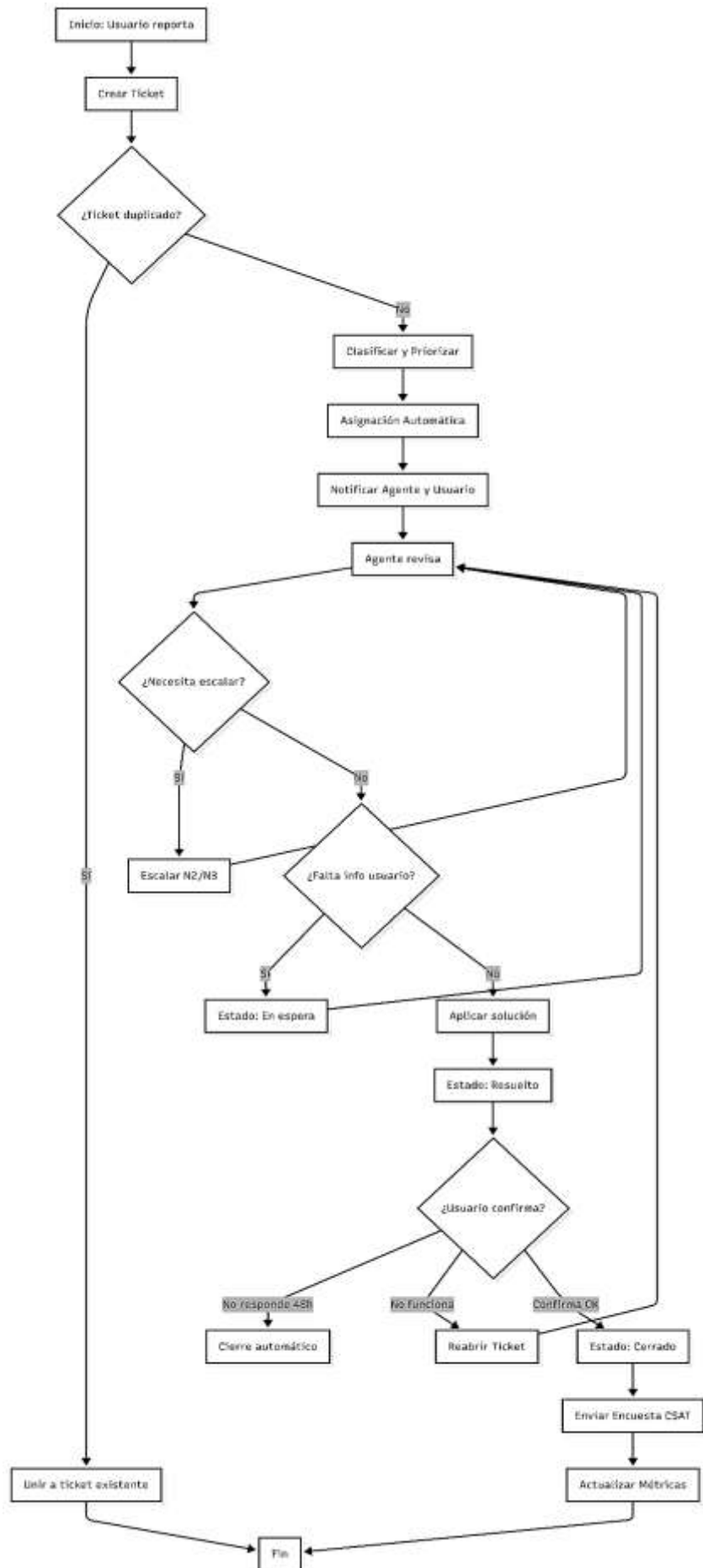
## Capturas de github











## Bibliografía

Falcon. (2020, 16 octubre). *Los 7 patrones de diseño de software más importantes*. DEV Community. <https://dev.to/gelopfalcon/los-7-patrones-de-diseno-de-software-mas-importantes-28l2>

Miriam, & Miriam. (2026, 24 febrero). *¿Qué son los patrones de diseño de software?* Profile Software Services. <https://profile.es/blog/patrones-de-diseno-de-software/>